

بسم الله الرحمن الرحيم

University College of Applied Science



DevOps

CI/CD Pipeline Deployment for Web or Mobile Application

Student preparation:

Malak Elyan.

Course Supervisor:

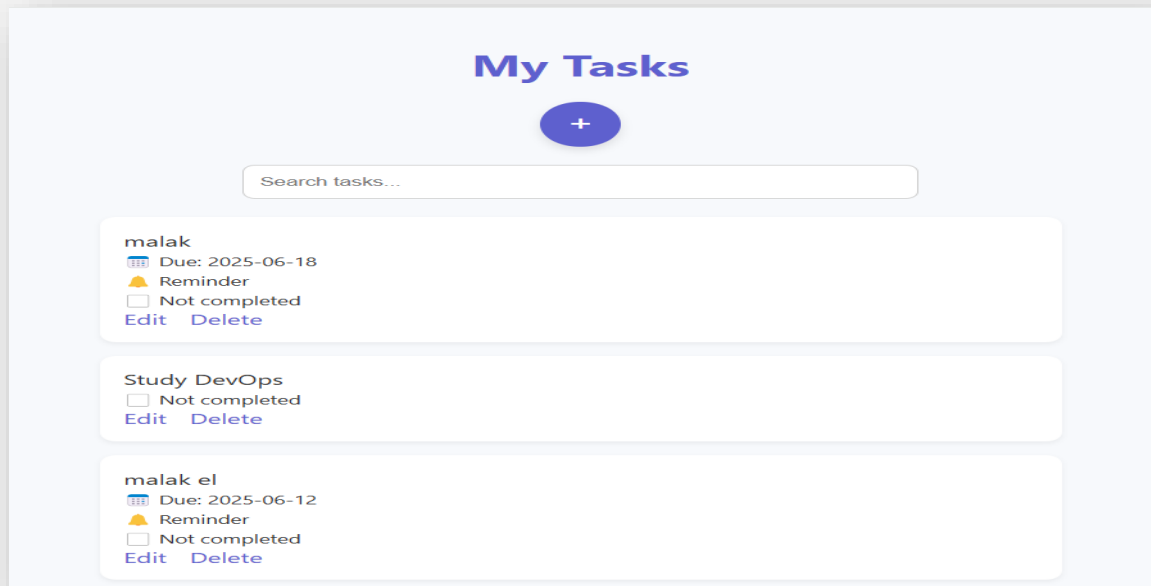
Eng. Raed Bolbol.

Introduction:

This project aims to apply DevOps concepts using tools such as Gitlab and Azure Devops, by building a simple web application using Flask and CI/CD pipeline application. The project includes building the application, its tests, creating a docker image, uploading it to Gitlab Container Registry, and then posting it on a hosting service (Render was used instead of the Azure App Service due to restrictions).

Basic Objectives:

- Build a Web Application Using Flask.
- Connect it to the PostgreSQL database.
- Preparing Gitlab CI/CD to perform Build, Test, Dockerize, Push, Deploy.
- Comparison of Azure Devops and Gitlab Devops.



Tools and Technologies:

1. **GitLab:** To Manage Code, Versions, CI/CD.
2. **Gitlab CI/CD:** to create automated pipeline.
3. **Docker:** to create an image of the container for the application.
4. **Gitlab Container Registry:** To store Docker's photos.
5. **Azure Boards:** To track tasks.

6. **Azure Pipelines:** to perform CI phases (publishing failed due to parallelism limitations).
7. **Render:** To host the app as a practical alternative to Azure App Service.



Project structure:

todo-app/

├─ app/

| └─ templates/

| └─ adit.html **Task modification form.**

| └─ index.html **Represents the main user interface**

| └─ __init__.py **Initialize flask application and organize the code.**

| └─ routes.py **Contains task management API endpoints.**

| └─ db.py **It is the file responsible for connecting to the database.**

└─ tests/

| └─ test_api.py **Automated tests to ensure that the API is working properly.**

└─ requirements.txt **List of libraries the project needs to run.**

└─ run.py **It is the running point of the project.**

└─ .gitlab-ci.yml **Definition of ci/cd stages in gitlab.**

└─ Dockerfile **The file is used to build a Docker image for the application.**

└─ render.yml **This is the settings for publishing your app on Render.com.**

└─ README.md **Documentation file explaining the project, its benefits.**

The function of each file:

1. run.py

Purpose: Entry-point script that starts the Flask server.

It imports `create_app ()` from `app/__init__.py`, creates the app object, and calls `app.run ()` (or is invoked by Gunicorn inside Docker).

2. app/__init__.py

Purpose: Application factory.

`create_app ()` instantiates the Flask app, registers the blueprint defined in `routes.py`, and (optionally) calls `init_db ()` to make sure the database table exists.

3. app/routes.py

Purpose: Holds all HTTP routes (endpoints) for the to-Do API and HTML views.

Functions inside this file handle:

- listing tasks,
- adding a task,
- editing / deleting a task,
- returning JSON for API calls.

4. requirements.txt

Purpose: Pinpoint every Python dependency the project needs—e.g. Flask, psycopg2-binary, gunicorn.

Install with:

```
pip install -r requirements.txt
```

5. tests/test_api.py

Purpose: Unit tests written with Python unittest.

Verifies that the API returns correct status codes and JSON responses.

Executed automatically in the test stage of the GitLab CI pipeline.

6. .gitlab-ci.yml

Purpose: Defines the complete GitLab CI/CD pipeline:

1. build – install Python packages

2. test – run unit tests
3. docker build – build a Docker image
4. push_to_registry – push that image to GitLab Container Registry
5. deploy – spin up the container (locally or on Render)

7. README.md

Purpose: Project documentation—what the app does, how to set it up locally, pipeline status badges, and any troubleshooting notes.

8. templates/index.html

Purpose: Main UI page: lists all tasks, shows due dates / reminders / completion status, and contains buttons to edit or delete tasks.

9. templates/edit.html

Purpose: HTML form used to edit an existing task (description, due date, reminder, completed flag).

10. Dockerfile

Purpose: Describes how to build the container image.

- Starts from an official python:3.10-slim base,
- copies project files,
- installs requirements.txt,
- sets the default CMD to run Gunicorn with the Flask app.

11. render.yaml

Purpose: Render.com service descriptor.

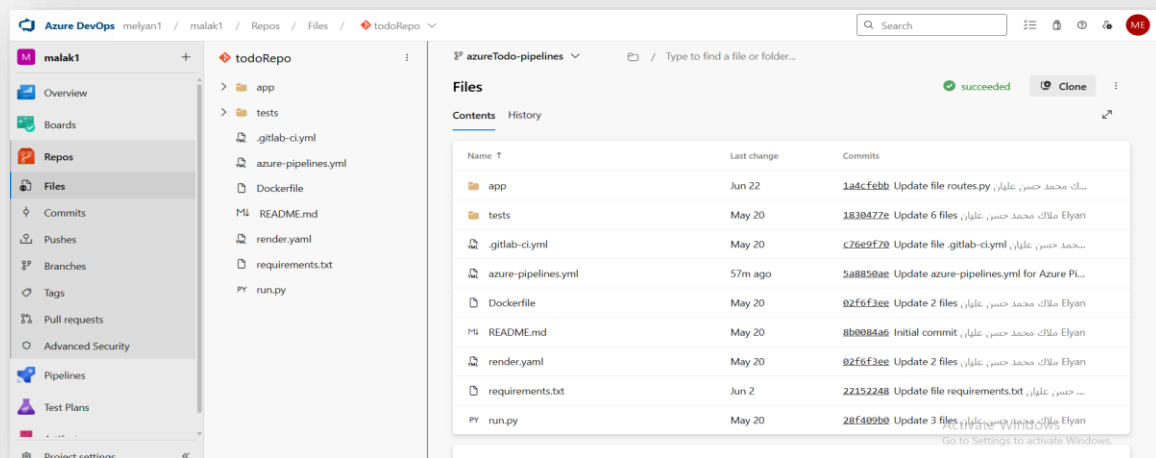
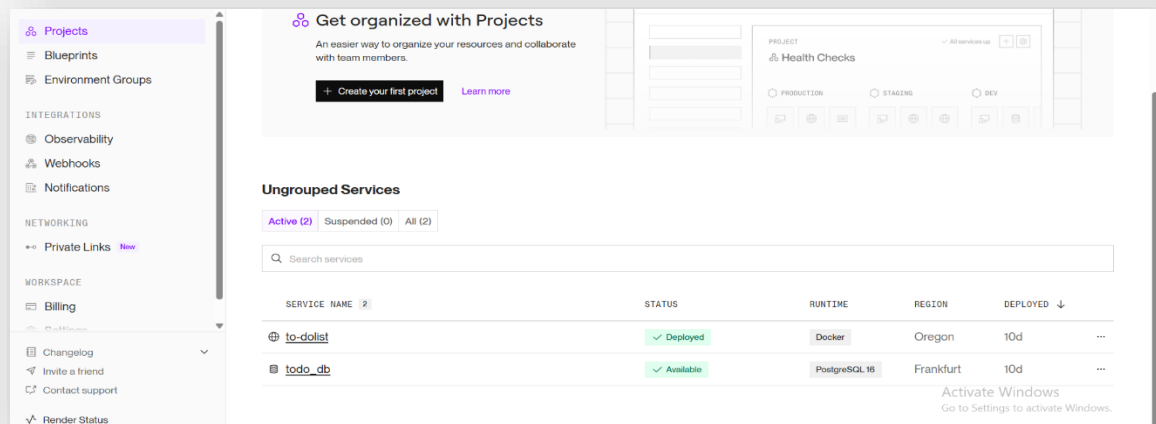
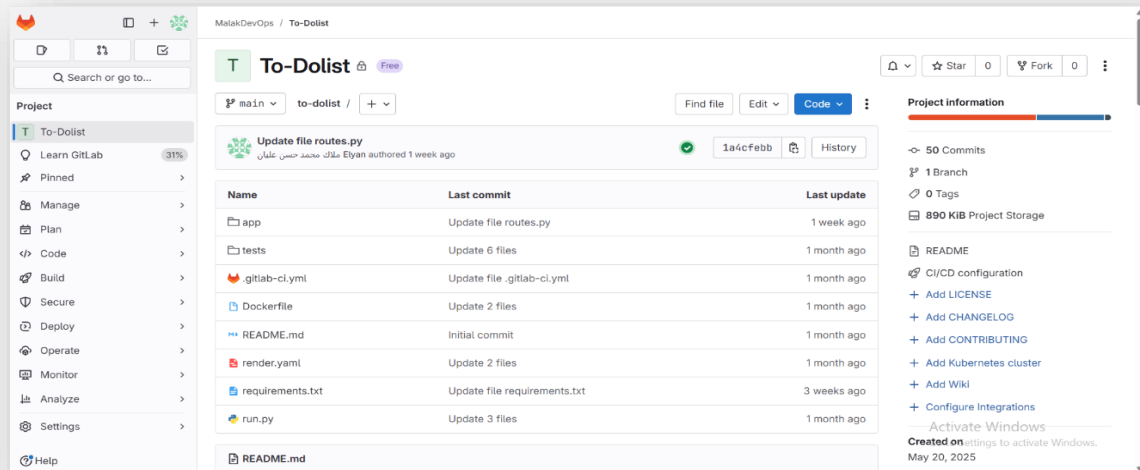
Specifies service name, runtime = Docker, region, and any environment variables Render should inject at deploy time.

12. app/db.py

Purpose: Low-level database helper module.

- get_db_connection () reads DATABASE_URL from the environment and returns a live psycopg2 connection.

- `init_db ()` creates the todos table if it doesn't exist.



CI/CD Pipeline Explanation:

We created the .gitlab-ci.yml file in the root directory of the project to define the CI/CD pipeline using GitLab CI. This file contains five main stages executed sequentially:

1. Build (Dependency Installation)

In this stage, all the required packages listed in requirements.txt are installed using:

```
pip install -r requirements.txt
```

2. Test (Unit Testing)

We run unit tests using the unittest library with the command:

```
python -m unittest discover -s tests -p "test_*.py"
```

This stage initially failed due to:

- Missing the DATABASE_URL environment variable.
- Database initialization errors.

We solved it by using Azure Variable Group to securely pass the database URL to the environment.

3. Docker Build

A Docker image is built from the application using the Dockerfile with the command:

```
docker build -t $CI_REGISTRY_IMAGE: $CI_COMMIT_SHORT_SHA.
```

4. Push to GitLab Container Registry

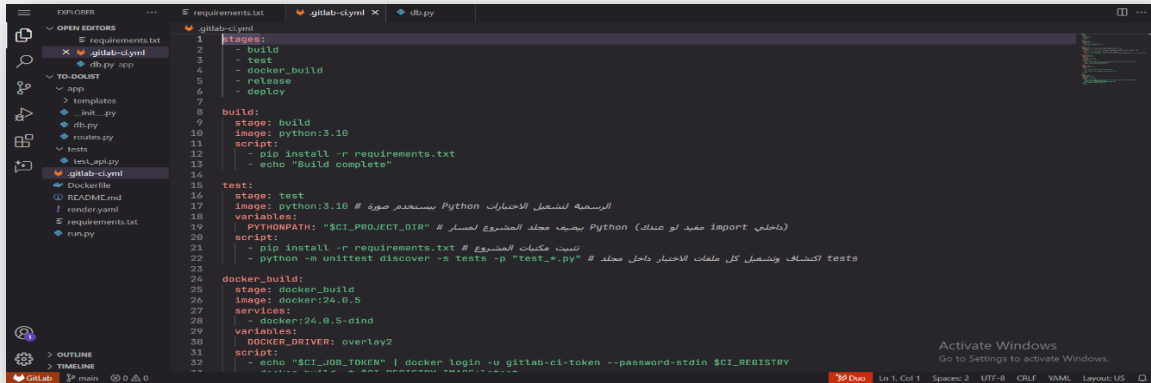
The image is pushed to GitLab's built-in container registry after logging in using:

```
docker push $CI_REGISTRY_IMAGE: $CI_COMMIT_SHORT_SHA
```

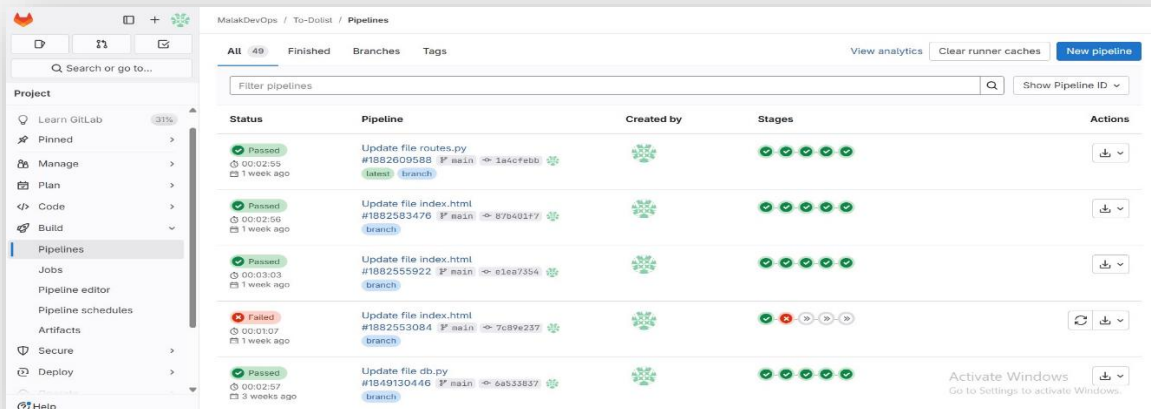
5. Deployment (Render)

Instead of Azure App Service, we used Render.com due to account limitations. We created a new web service on Render, linked it with the Docker image URL from GitLab Registry, and the app was deployed successfully. It auto-updates on every new push.

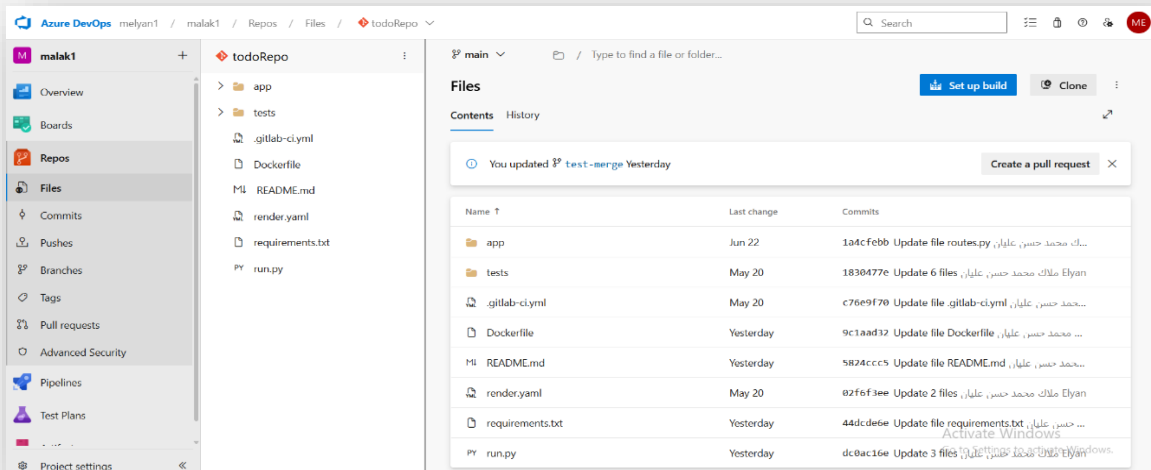
📌 Most stages succeeded after fixing early issues like YAML formatting and missing variables, and all steps were documented with pipeline logs and Render dashboard screenshots.



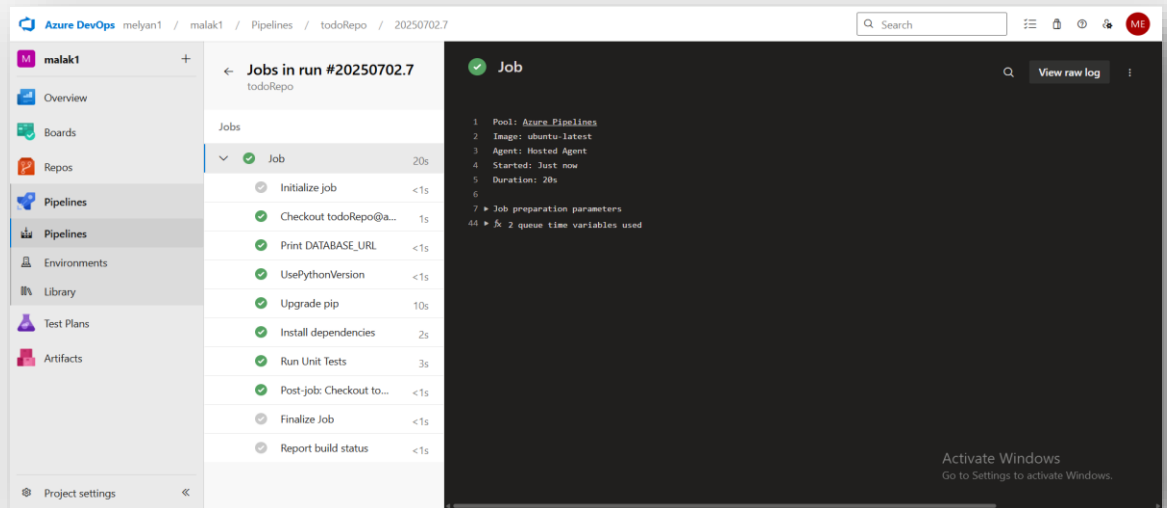
```
1 stages:
2   - build
3   - test
4   - docker_build
5   - release
6   - deploy
7
8 build:
9   stage: build
10  image: python:3.10
11  script:
12    - pip install -r requirements.txt
13    - echo "Build complete"
14
15 test:
16   stage: test
17   image: python:3.10 # الرئيسية لتشغيل الاختبارات Python باستخدام صورة
18   variables:
19     PYTHONPATH: "$CI_PROJECT_DIR" # يضيف مجلد المشروع لمسار Python (داخل import قيد لو عندك)
20   script:
21     - pip install -r requirements.txt # تثبيت مكتبات المشروع
22     - python -m unittest discover -s tests -p "test*.py" # تكتشف وتشغيل كل ملفات الاختبار داخل مجلد
23
24 docker_build:
25   stage: docker_build
26   image: docker:24.0.5
27   services:
28     - docker:24.0.5-dind
29   variables:
30     DOCKER_DRIVER: overlay2
31   script:
32     - echo "$CI_JOB_TOKEN" | docker login -u gitlab-ci-token --password-stdin $CI_REGISTRY
```



Status	Pipeline	Created by	Stages	Actions
Passed	Update file routes.py #188200588 1 week ago	1a4cfebb	latest branch	📄
Passed	Update file index.html #1882583476 1 week ago	879401f7	branch	📄
Passed	Update file index.html #1882555922 1 week ago	e1ea7354	branch	📄
Failed	Update file index.html #1882553084 1 week ago	7c89e237	branch	🔄 📄
Passed	Update file db.py #1849130446 3 weeks ago	6a535837	branch	📄

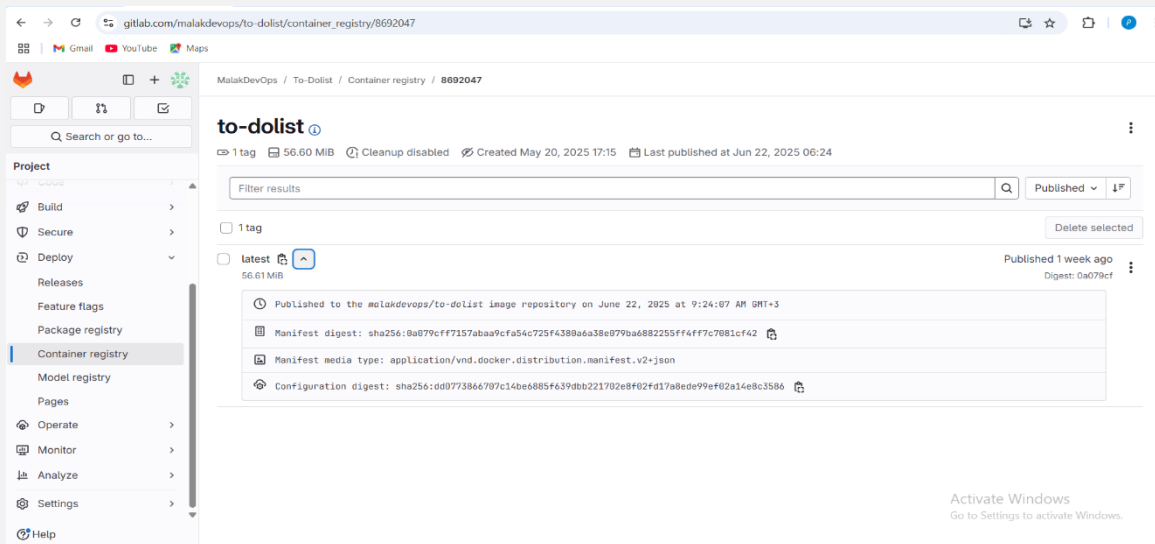


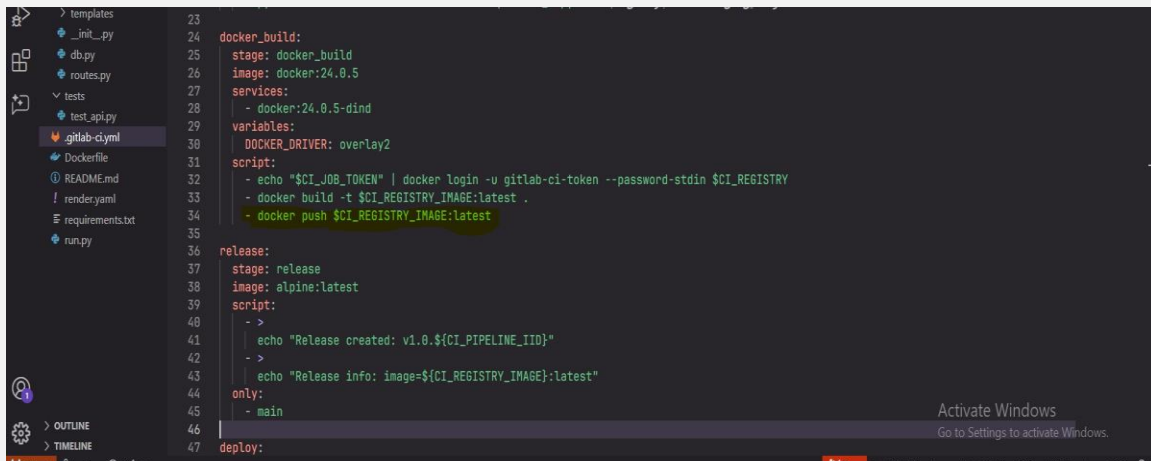
Name	Last change	Commits
app	Jun 22	1a4cfebb Update file routes.py
tests	May 20	1830477e Update 6 files
.gitlab-ci.yml	May 20	c76e9f70 Update file .gitlab-ci.yml
Dockerfile	Yesterday	9c1aad32 Update file Dockerfile
README.md	Yesterday	5824ccc5 Update file README.md
render.yaml	May 20	02f6f3ee Update 2 files
requirements.txt	Yesterday	44dcde6e Update file requirements.txt
run.py	Yesterday	dc9ac16e Update 3 files



Setting Docker and Gitlab Container Registry:

- Dockerfile was created to determine how the application image is built.
- We used Gitlab Container Registry as a place to save photos.
- We used Token Gitlab inside Pipeline to log in and pay photos.





Database Design:

The todos table in the PostgreSQL database was created programmatically through the `db.py` file using Python code. We didn't use any external DB management tools like pgAdmin or DBeaver, nor did we create the table manually. Instead, we used code-based schema creation for the following reasons:

Flexibility and Control: Using code allows us to initialize or recreate the table automatically, which is helpful during development and testing.

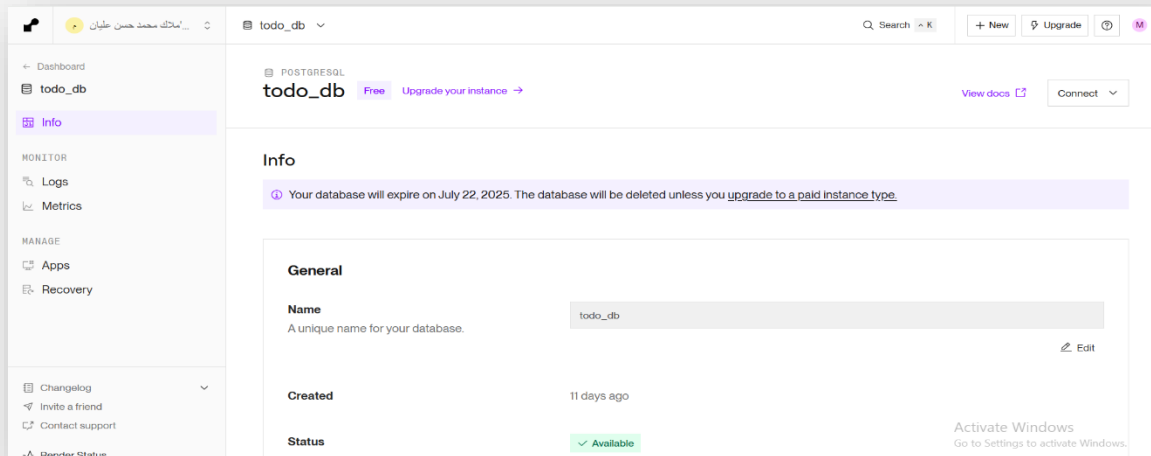
CI/CD Compatibility: During GitLab CI pipeline runs (e.g., testing or deployment), the table is created automatically without manual steps.

Easy to Modify: It's simpler to update the schema in code than to manually change it through a UI.

How It Was Done:

In the `db.py` file, we defined a function `init_db ()` which connects to the database using the `DATABASE_URL` environment variable and runs SQL code to create the todos table.

This function is called inside `create_app ()` in the `__init__.py` file, so the table is automatically created every time the app starts.



Deployment:

Why we used Render instead of Azure App Service:

During the implementation of our project, we faced issues trying to activate Azure for Students. Despite using a valid student email, Azure always returned the error:

“We couldn’t grant you an Azure for Students account. Please use a paid subscription.”

Additionally, the free Azure account has strict limitations for deploying apps. It often requires complex configurations like paid App Service Plans or Key Vault setup. So, we opted for a more practical and beginner-friendly alternative: Render.com.

Steps to Deploy on Render:

1. Creating a Web Service:

- We signed up at Render.com and created a new Web Service.

2. Connecting Docker Image from GitLab Container Registry:

- We chose "Deploy an existing image".
- We provided the image path from GitLab’s Container Registry (like `registry.gitlab.com/username/project:tag`).
- We configured the required environment variables including `DATABASE_URL`.

3. Setting Environment Variables:

- `DATABASE_URL`
- `PORT = 10000` (or appropriate port as needed)

Problems Faced During Deployment:

- Startup Delay: Render's free plan causes the service to "sleep" when inactive, and it takes around 50 seconds to start when accessed.
- Initial Errors: We had to ensure that the DATABASE_URL was properly set and that the application was listening on the right port.
- GitLab Access Token Issues: We generated a GitLab Access Token so Render could pull the Docker image from GitLab's Container Registry.

Azure Boards and Pipelines:

1. Using Azure Boards for Task Tracking:

We used Azure Boards to manage and track the progress of our project. We created multiple Work Items to represent each main task such as:

- Initializing the Flask project.
- Connecting to PostgreSQL database.
- Creating the Dockerfile.
- Setting up GitLab CI/CD pipeline.
- Deploying to Render.
- Writing the final report.

We organized these tasks using a Kanban Board, moving items through To Do → Doing → Done, which helped us stay organized and focused.

2. Attempting Azure Pipelines:

We tried to implement the same CI/CD pipeline using Azure Pipelines, writing a YAML file similar to the one used in GitLab.

However, we encountered a major issue:

Free Parallelism Restriction

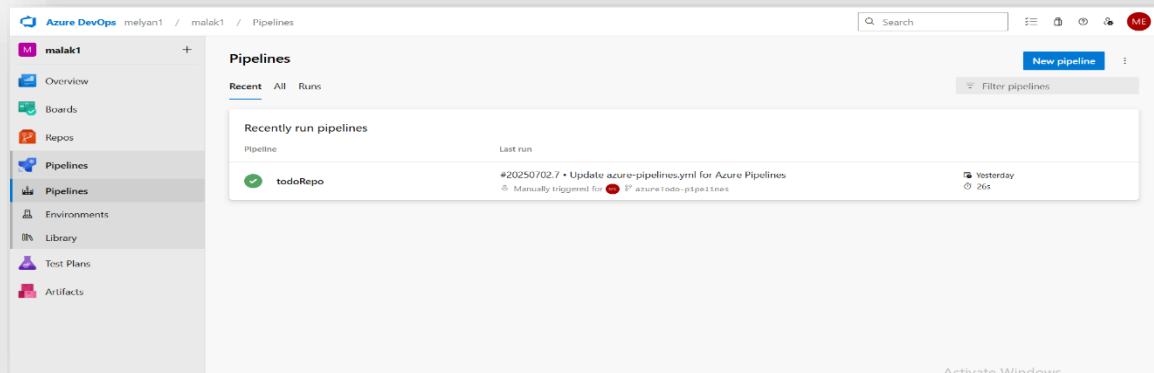
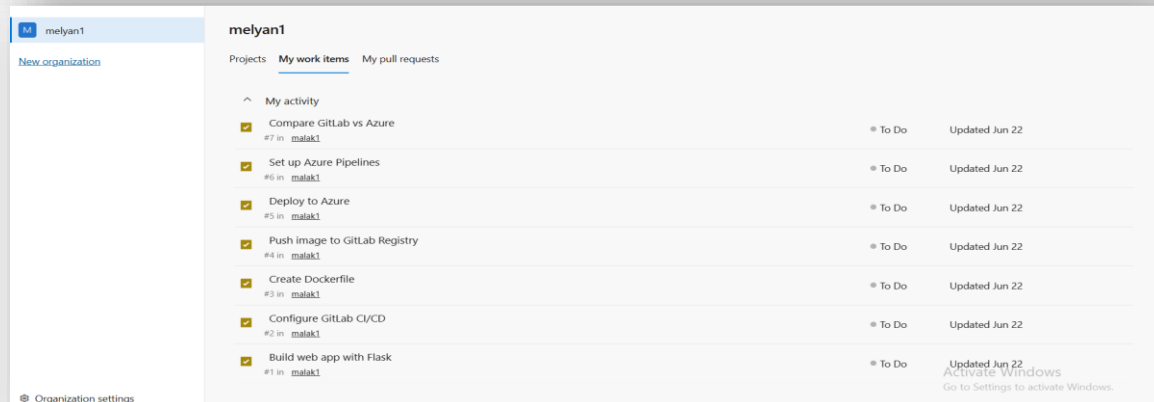
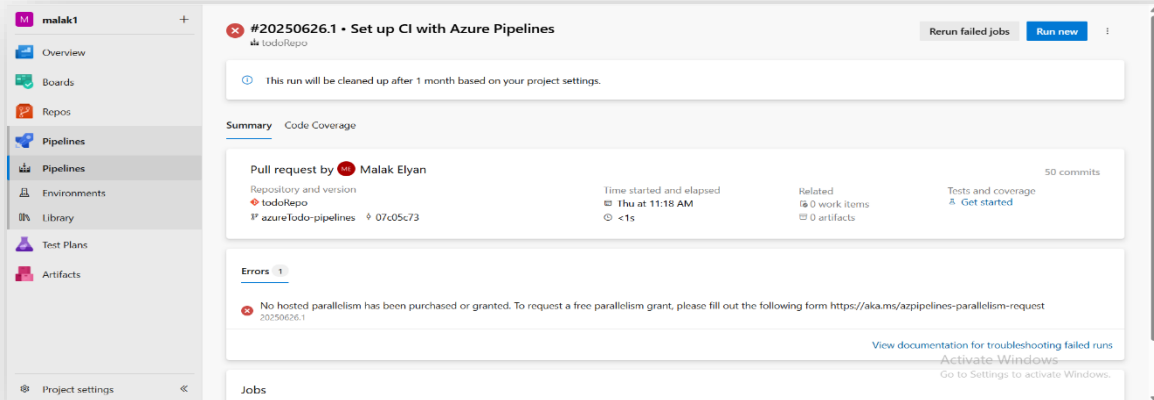
Our CI/CD jobs were blocked because Azure's free plan does not allow running parallel jobs. We were prompted to upgrade to a paid subscription.

As a result, the pipeline failed and we could not complete the CI/CD process on Azure.

3. Comparison: GitLab vs Azure

- In GitLab, all CI/CD stages (Build, Test, Dockerize, Deploy) succeeded smoothly without needing a paid plan.

- In Azure Pipelines, we could not execute the jobs due to free tier limitations.
- Therefore, we relied on GitLab for CI/CD, and used Azure mainly for Boards and experimentation with Pipelines.



Problems and Solutions:

1. Error: KeyError: 'DATABASE_URL'

Description: During the unit testing stage in GitLab CI, the error KeyError: 'DATABASE_URL' appeared, meaning the application couldn't access the database environment variable.

Cause: The environment variable DATABASE_URL was not set.

Solution: We created a variable group in Azure DevOps named todo-variables and linked it to the pipeline. In GitLab, we also made sure to pass DATABASE_URL correctly within the job script.

2. Azure Pipeline Failed Due to YAML Syntax Error

Description: The Azure pipeline failed to run because of a syntax error in the azure-pipelines.yml file.

Cause: Misaligned indentation or missing variable initialization.

Solution: We fixed the YAML indentation and defined the Python version explicitly ('3.10') instead of using an undefined variable.

3. Azure Pipelines Failed Due to Free Plan Limitations

Description: Azure Pipelines failed with a message about lacking the necessary parallelism or license (Azure for Students).

Cause: No access to a valid Azure for Students subscription.

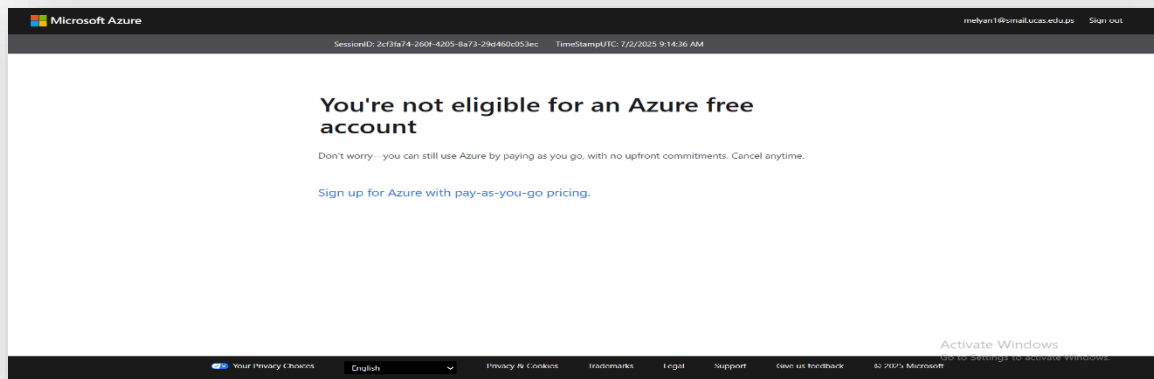
Solution: After failing to register for Azure for Students, we decided to fully switch to GitLab CI and use Render for deployment.

4. Error in requirements.txt

Description: Pip install failed due to incorrect or missing package names.

Cause: Some libraries were not written properly (missing versions or typos).

Solution: We corrected the requirements.txt file and re-installed the packages successfully.



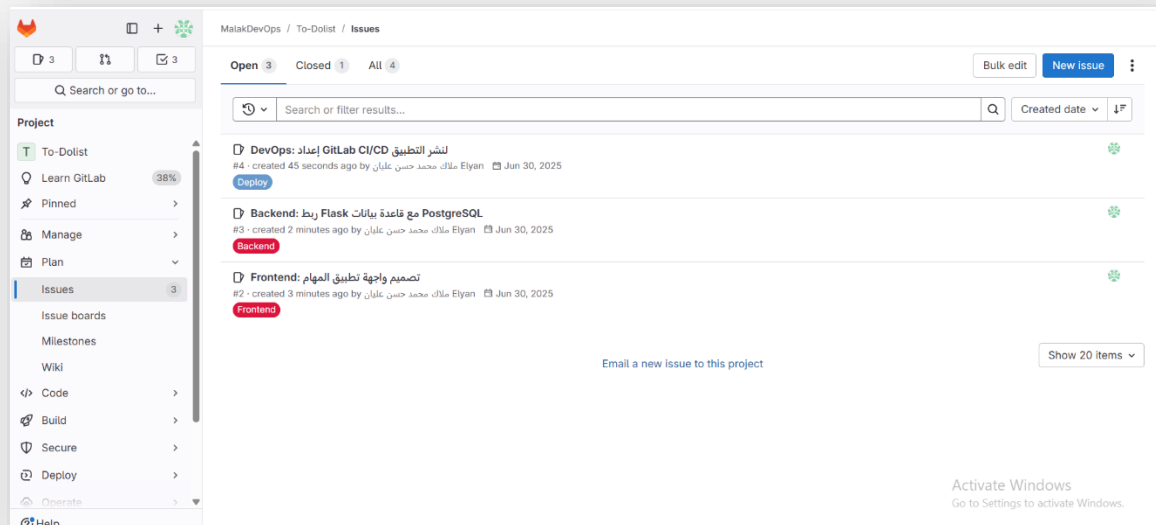
Comparison between gitlab devops and azure devops:

Feature	GitLab DevOps	Azure DevOps
Version Control	Excellent and easy to use. Fully integrated with the platform.	Powerful and smooth with advanced tools and branching strategies.
Task Management	Simple but limited. Built-in Issues feature is basic.	Fully integrated and powerful via Azure Boards . Suitable for large teams
CI/CD	Full CI/CD pipeline is easy to configure and runs smoothly.	Full-featured, but free-tier limits restrict parallel jobs and pipelines.
Docker Integration	Built-in GitLab Container Registry makes it seamless to use Docker.	No free built-in registry. Requires external configuration and setup.
Ease of Use	Unified interface. Everything in one place.	Tools are separated (Boards, Repos, Pipelines), which need manual linking.
Cost	Free for most features, even for private repositories.	Free tier is limited. Many features (like parallel jobs) require paid plans.
Documentation & Support	Active community and helpful documentation.	Strong official documentation and support from Microsoft

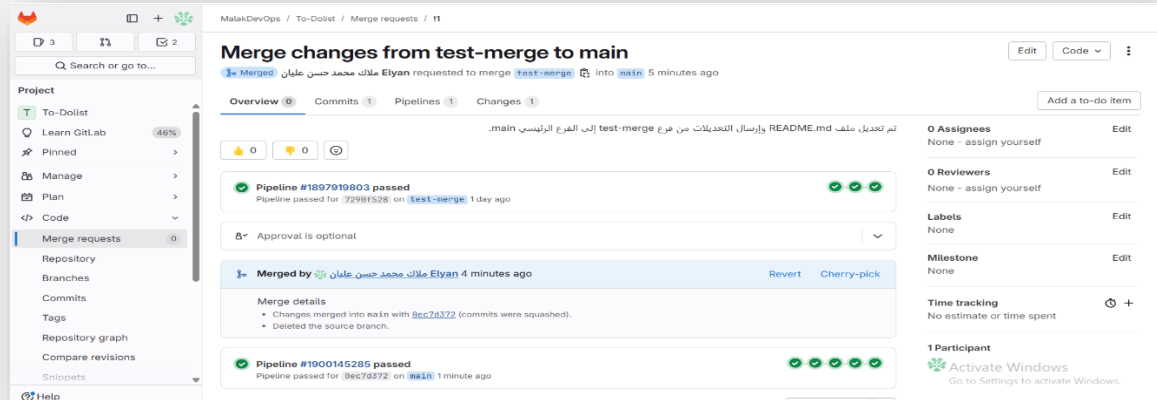
Our Experience:

- **We started the project with both tools and used GitLab for core CI/CD operations, including:**
 1. Building the Flask app.
 2. Running unit tests.
 3. Creating a Docker image.
 4. Pushing to GitLab's built-in Container Registry.
 5. Deploying to Render.
- **On Azure, we used Azure Boards for task tracking and attempted Azure Pipelines:**
 1. The CI steps worked.
 2. Deployment failed due to the free-tier parallelism limitations.
 3. Additionally, we were unable to access Azure for Students, and the free account did not allow full publishing, which forced us to switch to Render as an alternative.

Issues:



Merge Request:



This screenshot shows a Merge Request in GitLab titled "Merge changes from test-merge to main". The request is created by Elyan and targets the main branch. The interface includes a sidebar with project navigation, a main content area with an overview, commits, pipelines, and changes, and a right sidebar with assignees, reviewers, labels, milestones, and time tracking. The overview section shows that the pipeline #1897910803 passed and that the merge is optional. The changes section shows that the merge was performed by Elyan 4 minutes ago, with changes merged into main and the source branch deleted. The right sidebar shows 0 assignees, 0 reviewers, and 1 participant.

MalakDevOps / To-DoList / Merge requests / 11

Merge changes from test-merge to main

1 Merged by Elyan requested to merge test-merge into main 5 minutes ago

Overview 0 Commits 1 Pipelines 1 Changes 1

0 Assignees None - assign yourself Edit

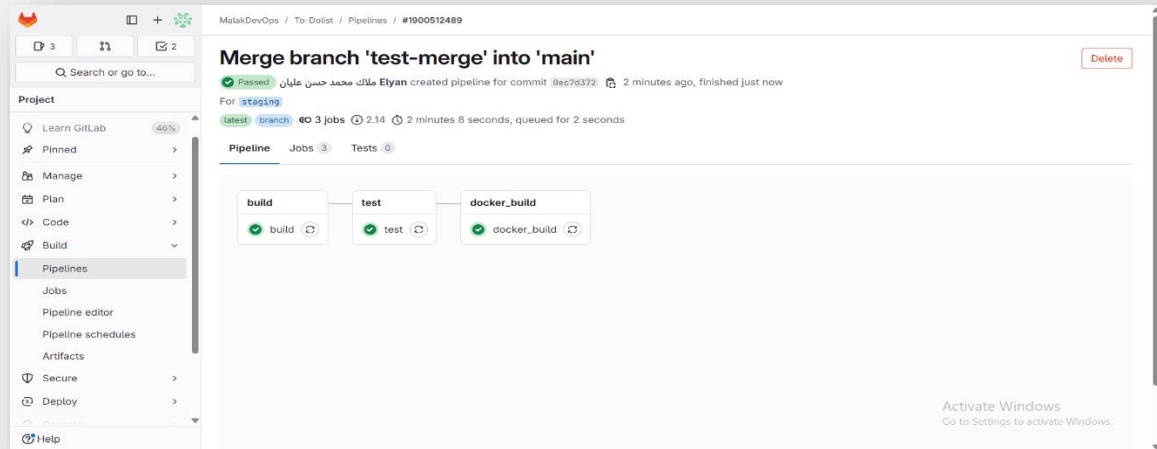
0 Reviewers None - assign yourself Edit

Labels None Edit

Milestone None Edit

Time tracking No estimate or time spent +

1 Participant Activate Windows Go to Settings to activate Windows.



This screenshot shows a Pipeline in GitLab titled "Merge branch 'test-merge' into 'main'". The pipeline is created by Elyan and targets the main branch. The interface includes a sidebar with project navigation, a main content area with an overview, jobs, and tests, and a right sidebar with assignees, reviewers, labels, milestones, and time tracking. The overview section shows that the pipeline is passed and that the merge was performed by Elyan 2 minutes ago. The jobs section shows that the pipeline consists of three jobs: build, test, and docker_build, all of which passed. The right sidebar shows 0 assignees, 0 reviewers, and 0 participants.

MalakDevOps / To-DoList / Pipelines / #1900512489

Merge branch 'test-merge' into 'main'

1 Passed Elyan created pipeline for commit @6c7d372 2 minutes ago, finished just now

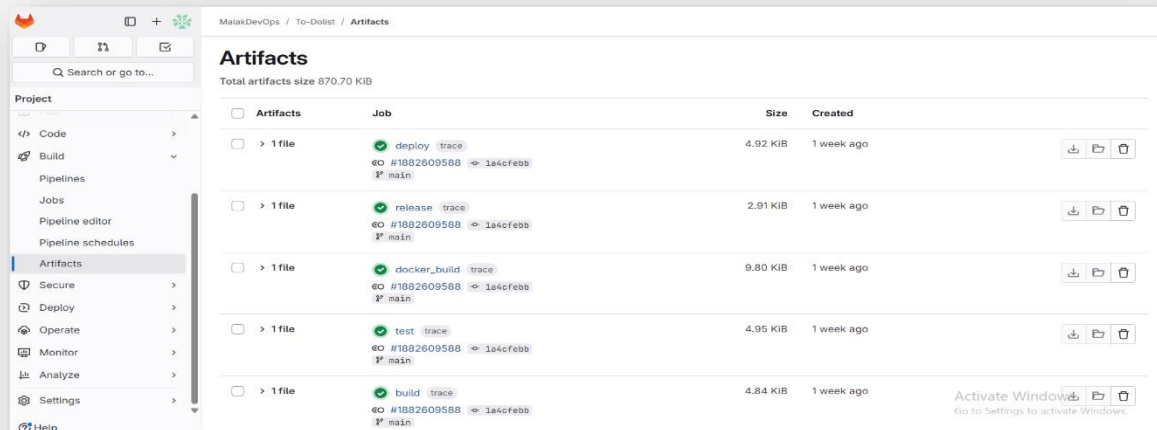
For staging latest branch 3 Jobs 2.14 2 minutes 8 seconds, queued for 2 seconds

Pipeline Jobs 3 Tests 0

build test docker_build

Activate Windows Go to Settings to activate Windows.

Artifacts:



This screenshot shows the Artifacts page in GitLab. The page displays a table of artifacts with columns for Artifacts, Job, Size, and Created. The table lists six artifacts, each with a download icon, a folder icon, and a trash icon. The artifacts are: 1 file (4.92 KIB, 1 week ago), 1 file (2.91 KIB, 1 week ago), 1 file (9.80 KIB, 1 week ago), 1 file (4.95 KIB, 1 week ago), and 1 file (4.84 KIB, 1 week ago). The right sidebar shows 0 assignees, 0 reviewers, and 0 participants.

MalakDevOps / To-DoList / Artifacts

Artifacts

Total artifacts size 870.70 KIB

Artifacts	Job	Size	Created
1 file	deploy trace eo #1882609588 1a4cfebb main	4.92 KIB	1 week ago
1 file	release trace eo #1882609588 1a4cfebb main	2.91 KIB	1 week ago
1 file	docker_build trace eo #1882609588 1a4cfebb main	9.80 KIB	1 week ago
1 file	test trace eo #1882609588 1a4cfebb main	4.95 KIB	1 week ago
1 file	build trace eo #1882609588 1a4cfebb main	4.84 KIB	1 week ago

Activate Windows Go to Settings to activate Windows.

Conclusion & Future Enhancements:

Through this project, I gained hands-on experience in building full CI/CD pipelines using DevOps tools like GitLab and Azure DevOps. I developed a solid understanding of how to configure pipeline stages, use Docker to build and push container images, manage environment variables, and deploy applications on external platforms like Render. Additionally, this project helped me improve my task tracking and workflow organization using Azure Boards.

Despite the overall success, limitations such as the inability to access Azure App Service due to the lack of a student license led me to explore and implement alternative solutions, which greatly enhanced my adaptability and problem-solving skills.

If I had more time, I would have implemented the following improvements:

1. Set up multiple deployment environments such as Development, Staging, and Production for better testing and rollout management.
2. Use Azure Key Vault to securely store sensitive credentials and access keys instead of plain environment variables.
3. Enhance the user interface of the application to make it more appealing and user-friendly.
4. Add more testing coverage, including API and integration tests to ensure robustness.
5. Improve performance monitoring by integrating monitoring tools post-deployment.

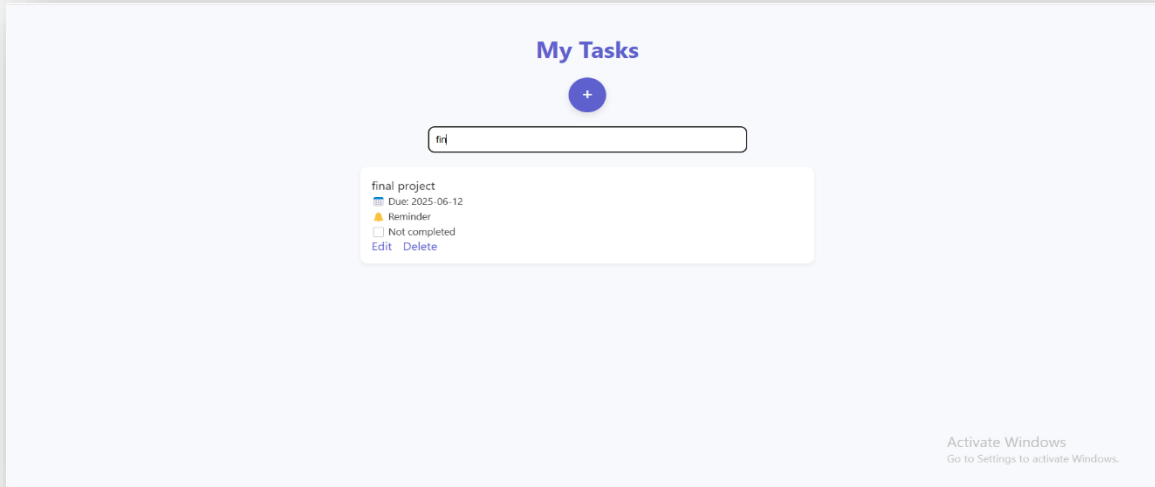
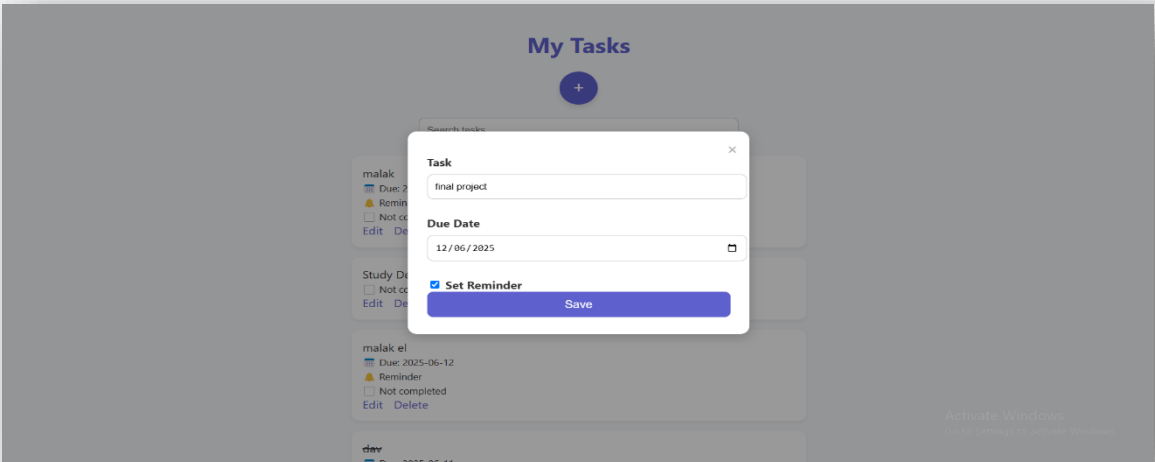
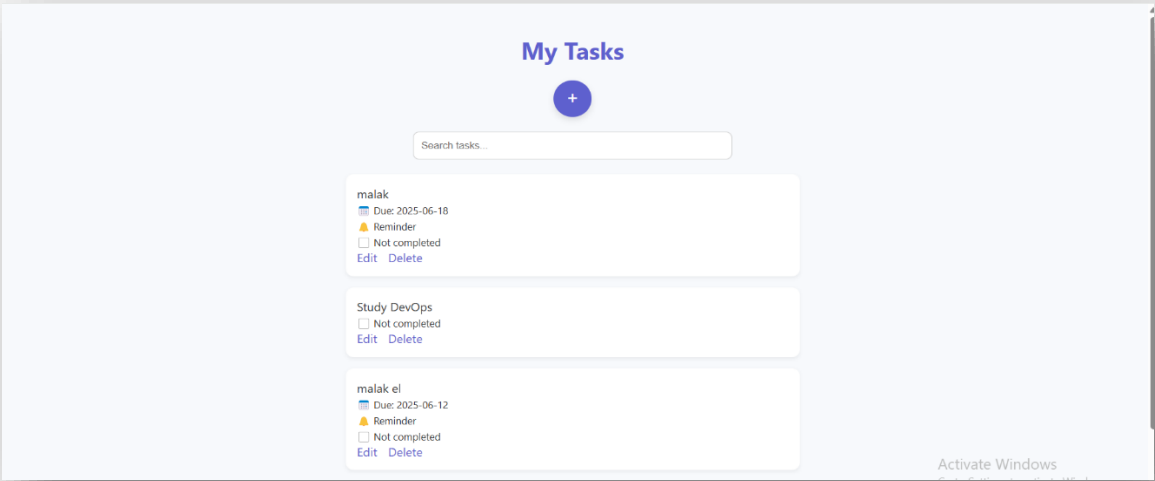
Links attached to the project:

Gitlab: <https://gitlab.com/malakdevops/to-dolist.git>

Azure: <https://dev.azure.com/melyan1/>

Render: <https://dashboard.render.com/d/dpg-d1boh8dl3ps73esr85g-a>

Project: <https://to-dolist-ii06.onrender.com>



Edit Task

Task

malak

Due Date

18/06/2025

☒ Set Reminder

Save

☐ Mark as Completed

Back to List

Activate Windows
Go to Settings to activate Windows.

My Tasks

+

mal

malak el

Due: 2025-06-12

Reminder

☐ Not completed

Edit Delete

malak

Due: 2025-04-23

Reminder

☒ Completed

Edit Delete

Activate Windows
Go to Settings to activate Windows.

My Tasks

+

Search tasks

Study DevOps

☐ Not completed

Edit Delete

malak el

Due: 2025-06-12

Reminder

☐ Not completed

Edit Delete

malak

Due: 2025-06-11

Reminder

☒ Completed

Edit Delete

final project

Due: 2025-06-11

Reminder

☒ Completed

Edit Delete

هل أنت متأكد من حذف هذه المهمة؟

إلغاء

نعم، احذف

Activate Windows
Go to Settings to activate Windows.

تم بحمد الله